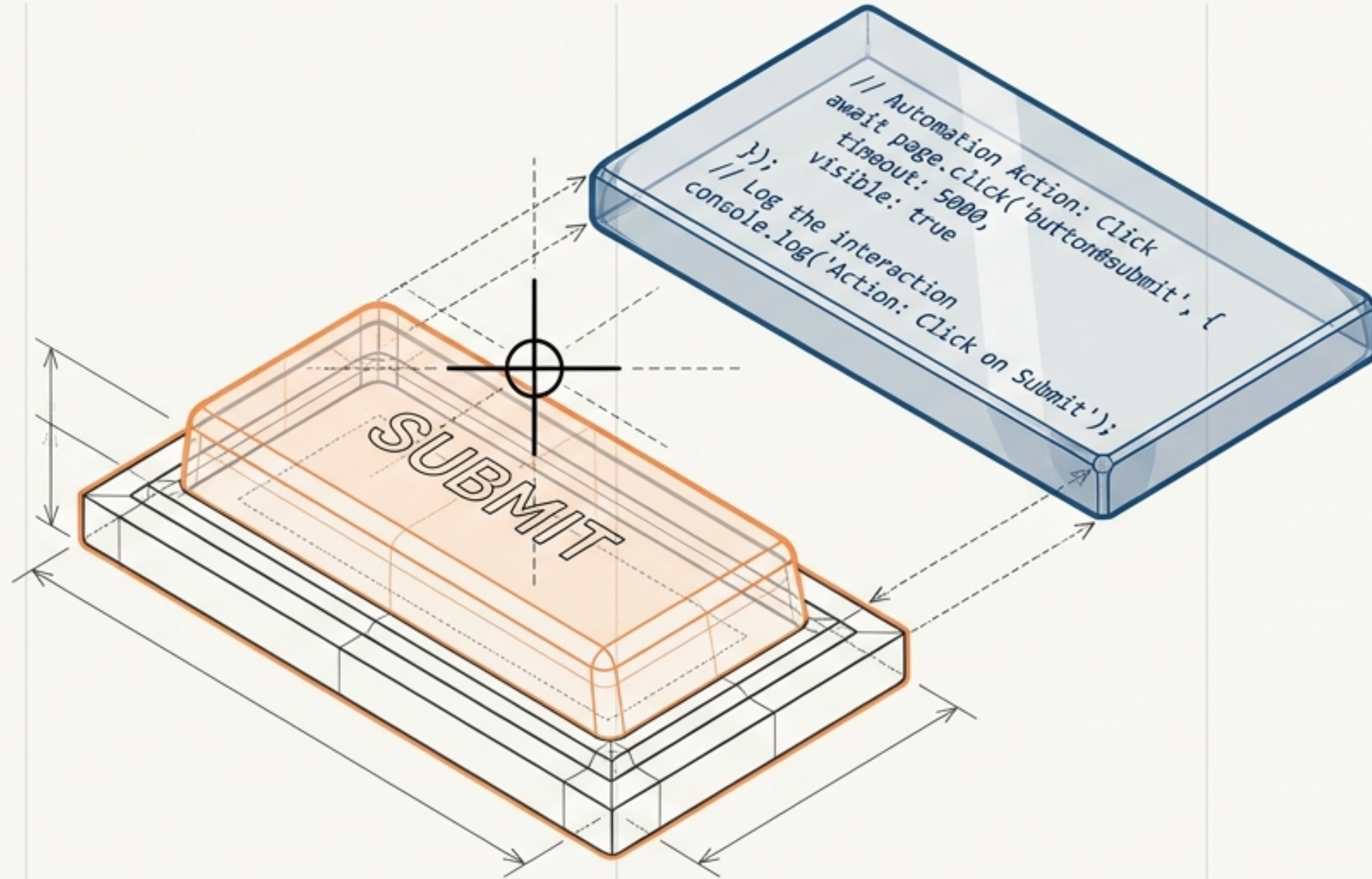


# The Architecture of a Click

Deconstructing the Systems Anatomy of an AI-Powered Test Automation Extension





# Bridging the Visual DOM and Test Automation Code

The extension eliminates manual DOM traversal by sitting as a translation layer between the browser and the LLM. Visual clicks instantly resolve to production-ready Selenium or Playwright logic.

## The Old Way

```
<div><ul><li><a  
  class='nav-item  
  menu-link active'  
  href='/login'>  
  <span>Login  
  </li></span></a></li>  
</ul>  
</div>
```

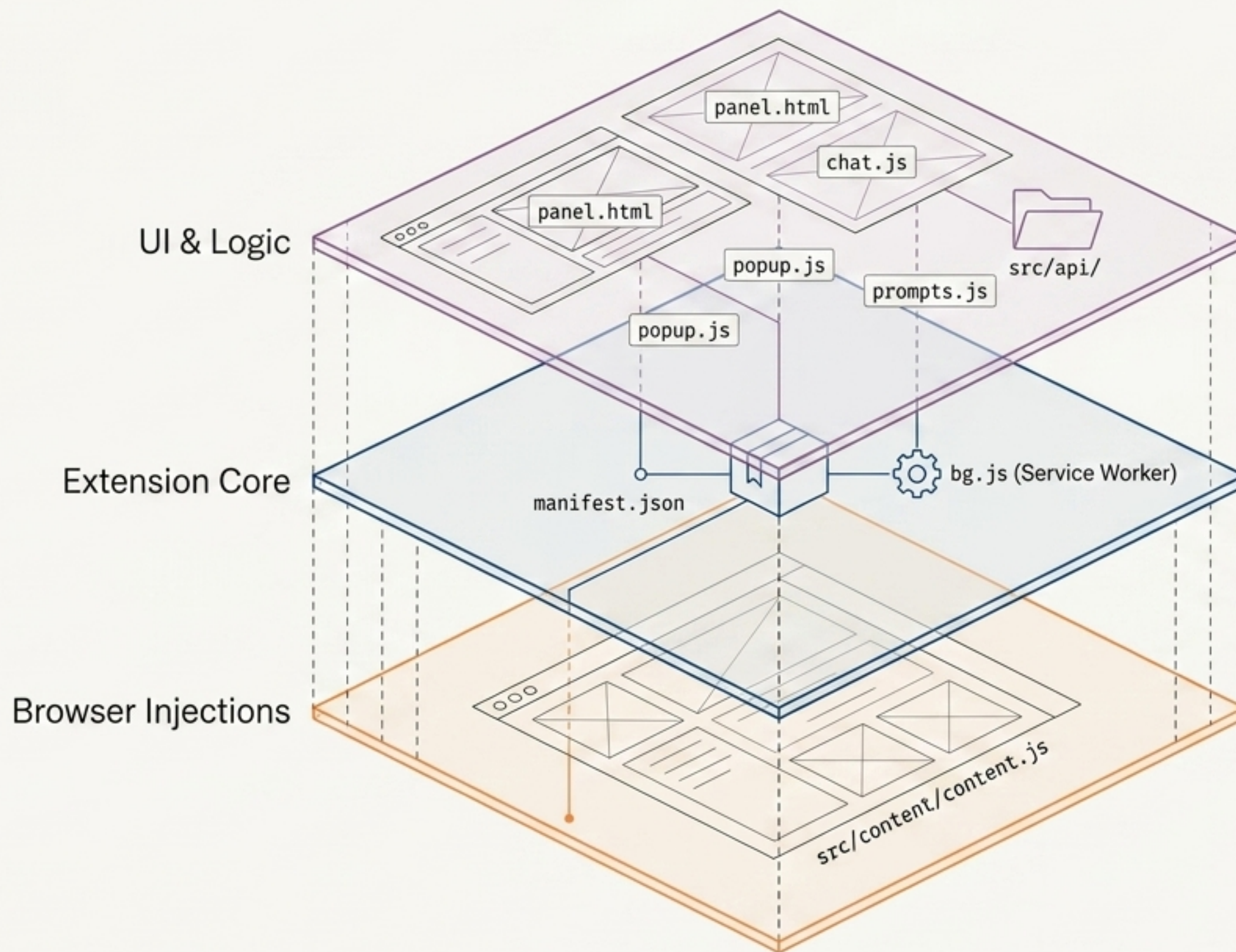
Login

## The Extension Way

```
await page.locator('.nav-item  
.menu-link.active').click();
```



# The Architectural Blueprint: File-to-Function Mapping

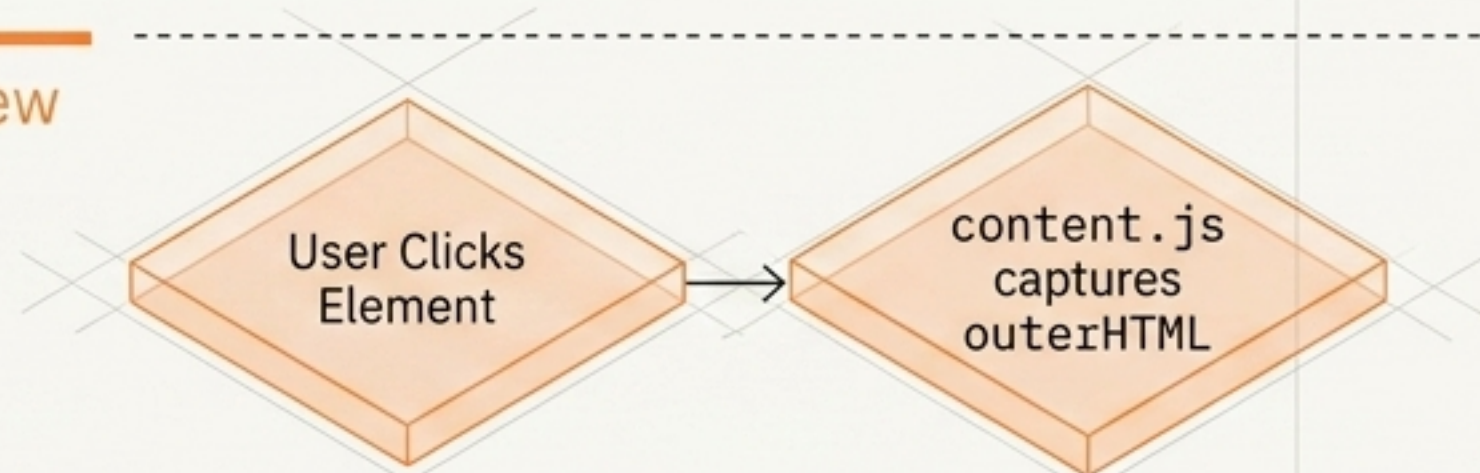


**Zero-Flash Rendering:**  
`src/config/appConfig.js` and `init-colors.js` execute instantaneously to inject CSS variables, preventing any white flash before the UI loads.

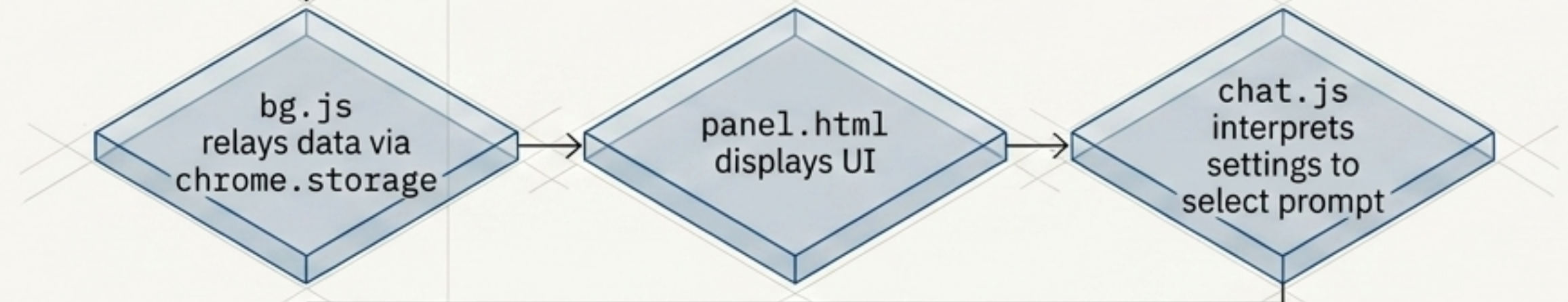


# The Data Pipeline: Tracing the Lifecycle of a DOM Element

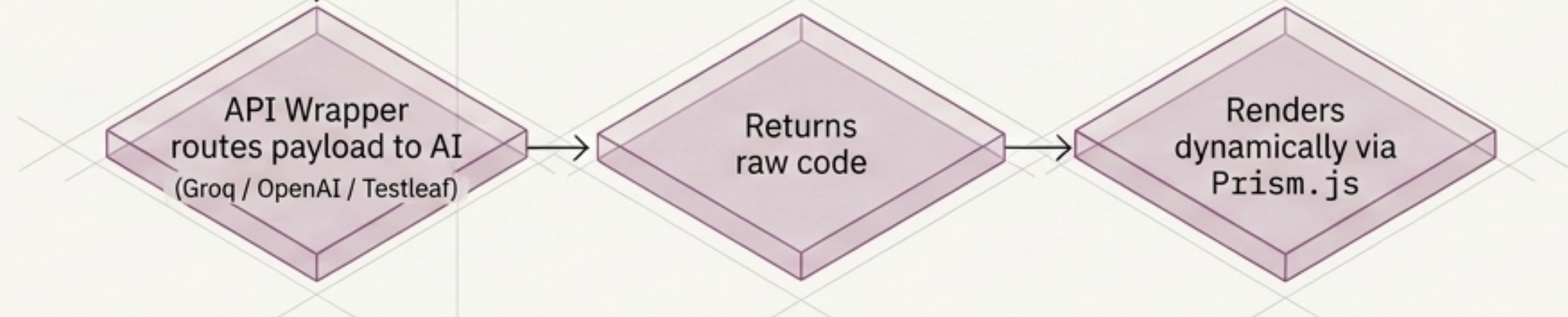
## Webpage View



## Extension Core

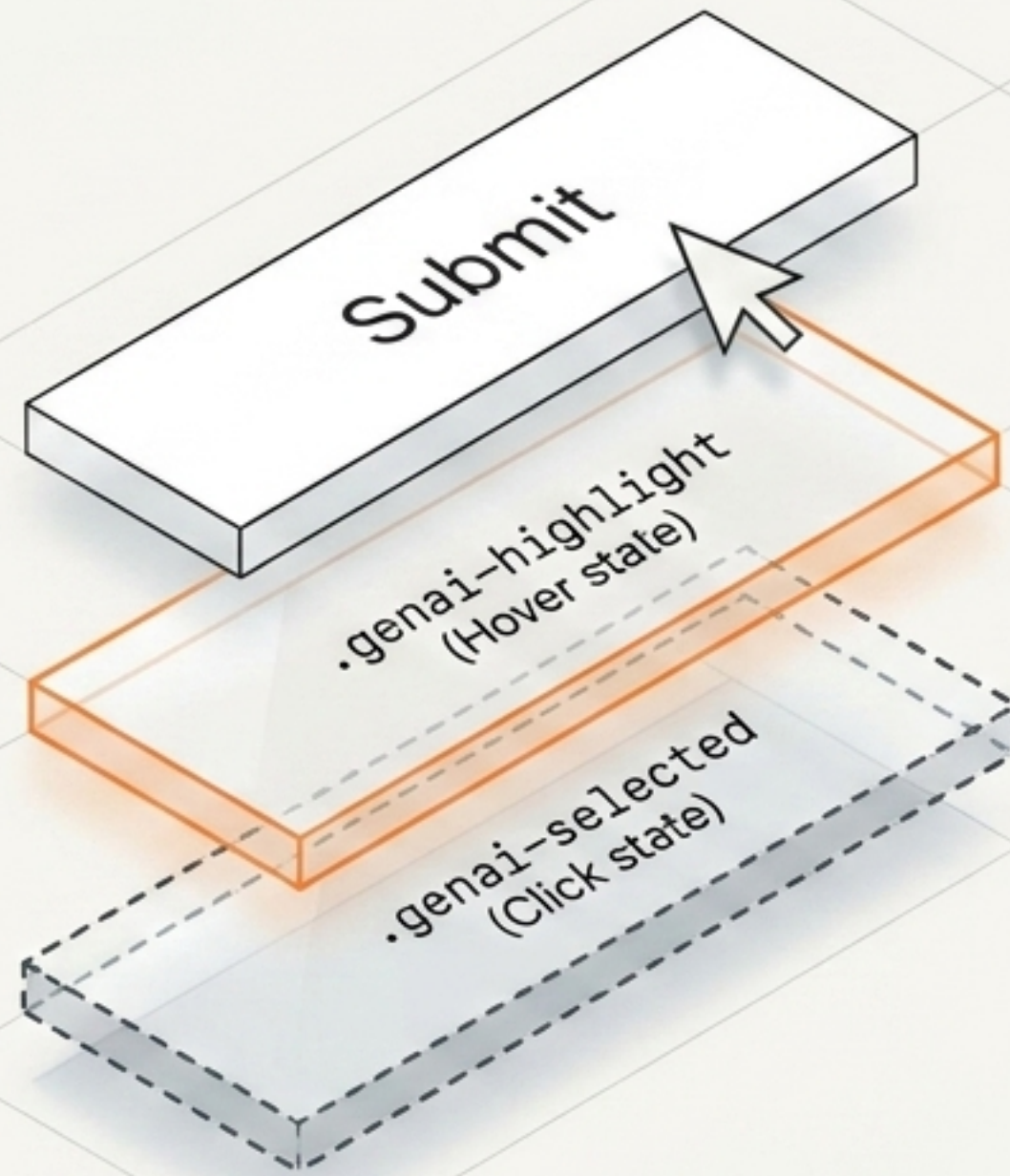


## Cloud API



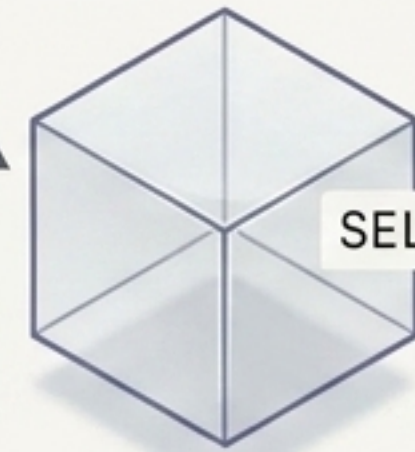


# The Inspector Engine: Visual Targeting and DOM Extraction



1. `content.js` injected into active tab.
2. Mouse hover/click toggles `.genai-highlight` and `.genai-selected` classes.
3. `chrome.runtime.sendMessage` fires `SELECTED_DOM_CONTENT` payload.
4. `bg.js` intercepts and writes the snippet to `chrome.storage.local`.

`chrome.runtime.sendMessage`

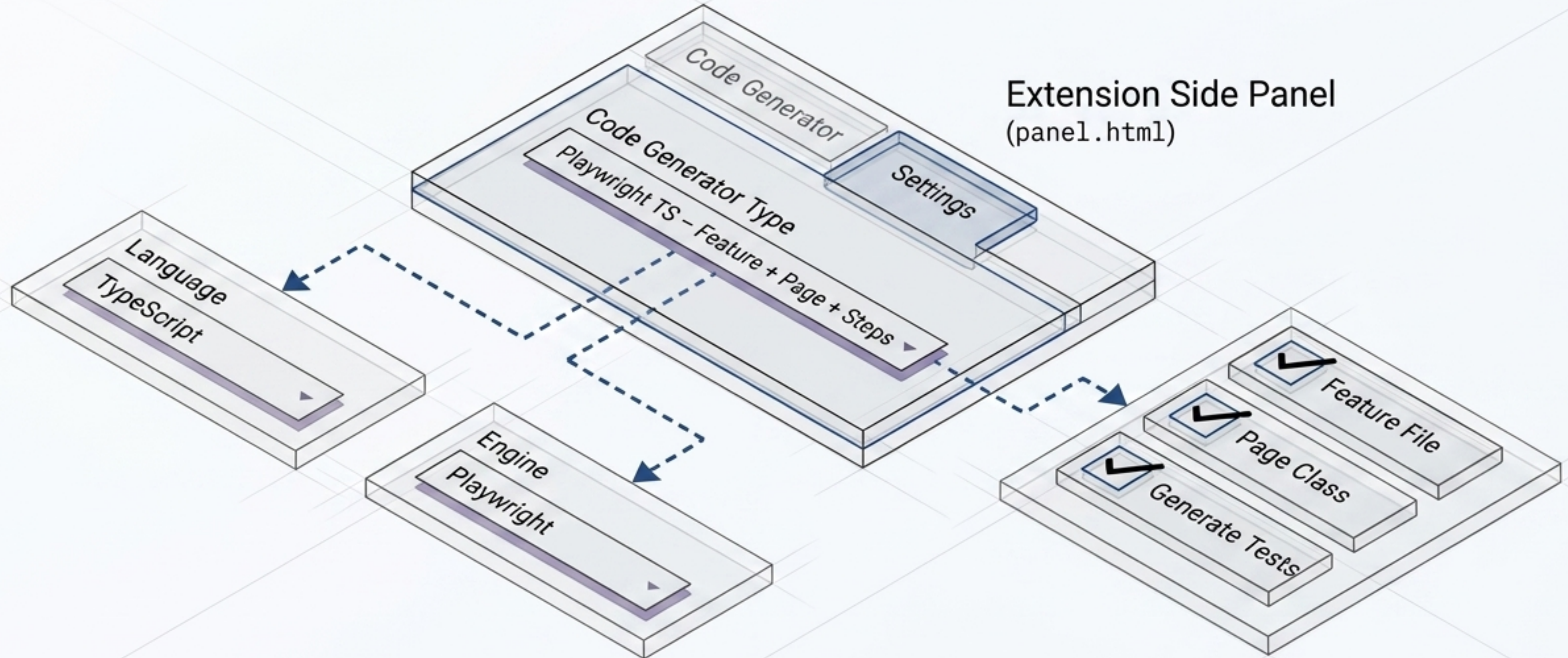


`SELECTED_DOM_CONTENT`



# UI & State Management: Frictionless Context Configuration

`src/scripts/popup.js` actively listens to `chrome.storage.sync`. When a Code Generator Type is selected, `applyCheckboxesForGeneratorType()` fires, instantly auto-mapping language, engine, and structural checkboxes.





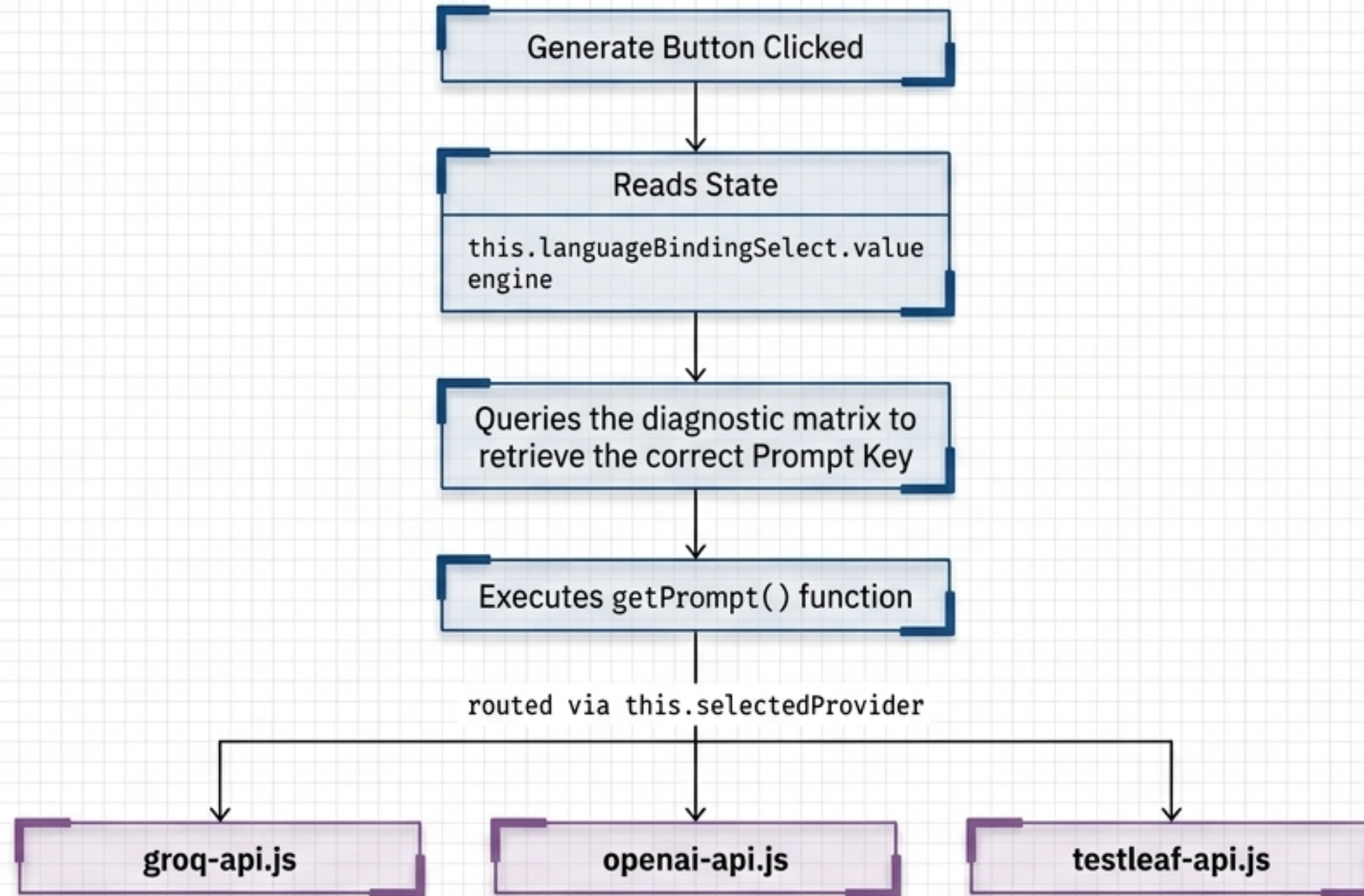
# The Prompt Routing Logic: `getPromptKeys(lang, eng)`

| User Checkbox State | Java + Selenium Engine            | TypeScript + Playwright Engine               |
|---------------------|-----------------------------------|----------------------------------------------|
| Feature only        | CUCUMBER_ONLY                     | PLAYWRIGHT_TYPESCRIPT_FEATURE_ONLY           |
| Page only           | SELENIUM_JAVA_PAGE_ONLY           | PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY              |
| Page + Tests        | SELENIUM_JAVA_PAGE_ONLY           | PLAYWRIGHT_TYPESCRIPT_WITH_TESTS             |
| Feature + Tests     | CUCUMBER_WITH_SELENIUM_JAVA_STEPS | PLAYWRIGHT_TYPESCRIPT_WITH_FEATURE_AND_TESTS |
| All 3               | SELENIUM_JAVA_ALL                 | PLAYWRIGHT_TYPESCRIPT_ALL                    |



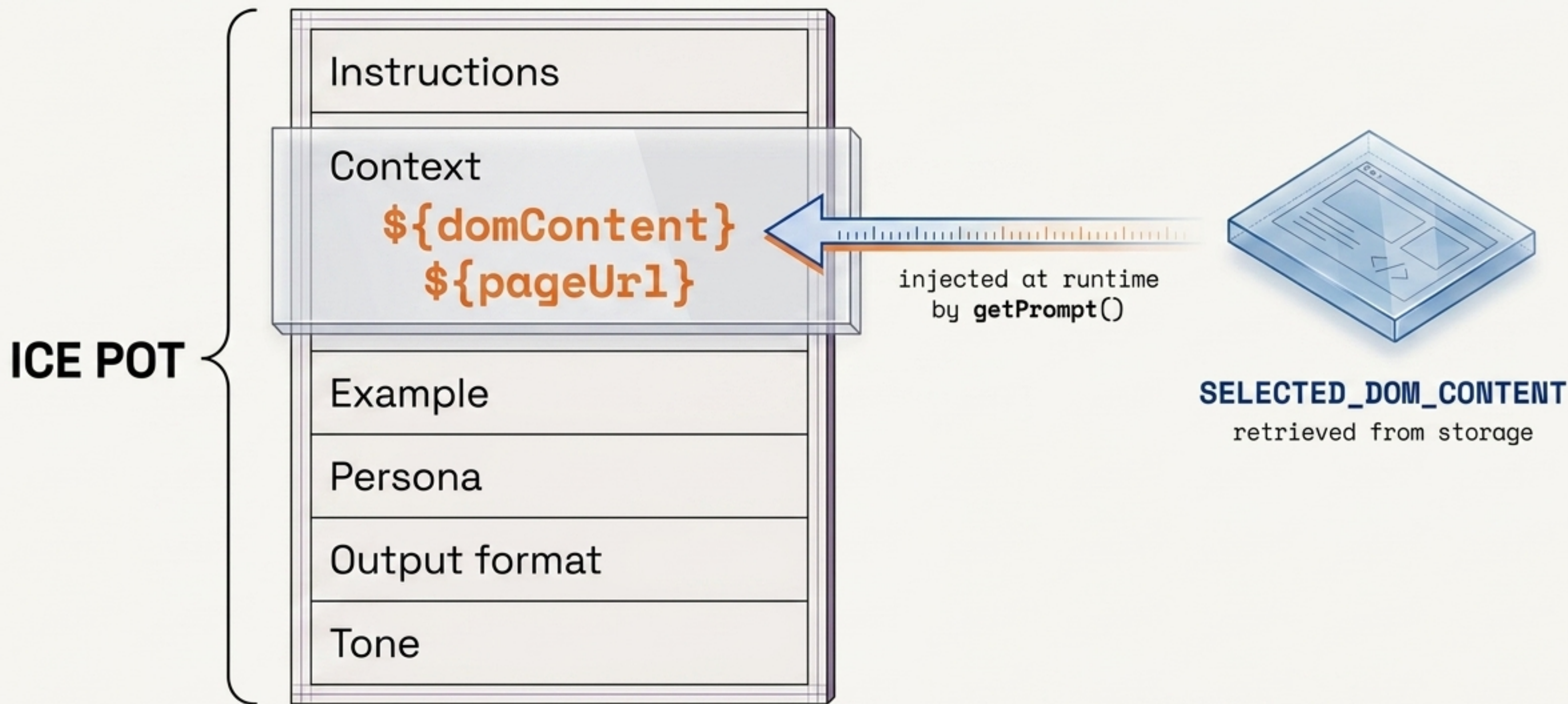
# Execution Flow: From “Generate” to the Cloud API

Once the AI returns the response, `chat.js` intercepts the payload and renders syntax-highlighted code directly in the side panel using `Marked` and `Prism.js`.





# Dynamic Prompt Engineering via the ICE POT Framework

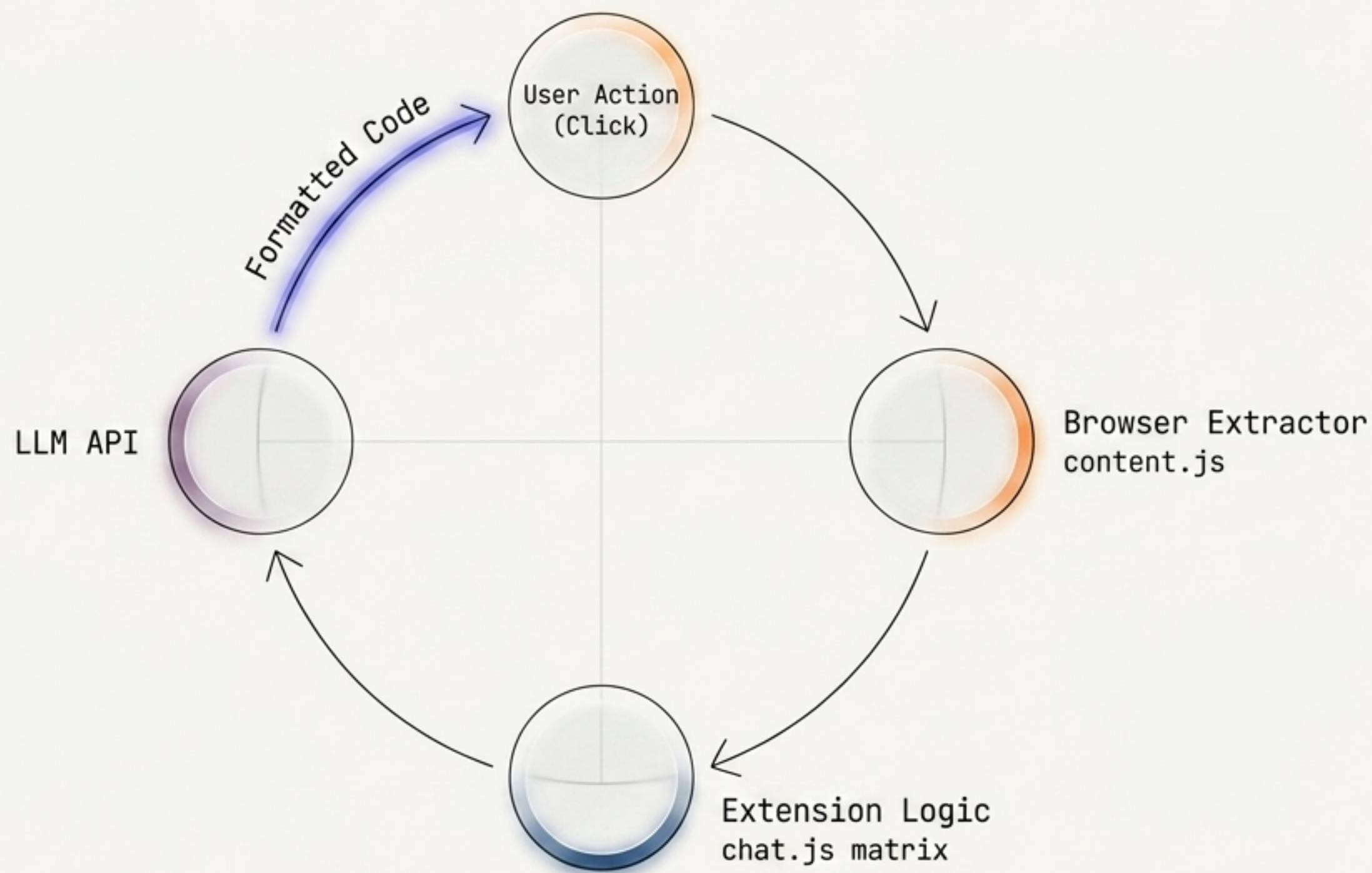








# System Summary: The Complete Click-to-Code Loop



Seamless integration from visual web inspection to production-ready automation code—eliminating friction, manual DOM traversal, and complex test boilerplate.